Checkpoint Two Report

Hisham Issa | 1194466 | hissa01@uoguelph.ca

Introduction

For this checkpoint, I added **semantic analysis** and **type checking** to my C- compiler. Building on the scanner and parser from checkpoint one, I implemented a symbol table to track declarations across nested scopes and a type checker to verify that operations, assignments, and function calls are semantically valid. The goal was to catch errors like using undeclared variables, type mismatches in expressions, and incorrect function calls. I used a visitor pattern to traverse the abstract syntax tree in post-order, building the symbol table and performing type checks in a single pass. The implementation handles all the semantic rules of C-, including nested scopes, array parameters, function signatures, and the predefined `input()` and `output()` functions. This report covers how I implemented the symbol table, my approach to type checking, the challenges I faced, and what I learned from the process.

Symbol Table Implementation

The symbol table is the foundation of semantic analysis. It tracks which names are declared, their types, and their scope. I used a `HashMap<String, ArrayList<NodeType>>` to map names to lists of declarations. This design naturally handles nested scopes because multiple declarations of the same name can coexist at different levels. Each `NodeType` entry contains three pieces of information: the variable name, a reference to its declaration node, and the scope level where it was declared. The scope level starts at 0 for global declarations, becomes 1 when entering a function, and increments for each nested block. This simple numbering scheme makes it easy to track which declarations are currently visible.

I implemented three core operations. **Insert**, **lookup**, and **delete**. Insert adds a new declaration to the front of the list for a given name, but first checks if that name already exists at the current scope level. If it does, that's a redefinition error. Lookup searches the list for a name and returns the first entry at or below the current scope level, implementing the "most closely nested" rule. Delete removes all entries at a specific level when exiting a scope.

Function parameters presented an interesting challenge. In C-, parameters and local variables belong to the same scope. My initial approach created a new scope when entering the function body, but this put parameters and local variables at different levels. I fixed this by incrementing the level before processing parameters, so both parameters and the function body's local variables end up at the same level.

Type Checking Implementation

Type checking verifies that operations and assignments use compatible types. I added a `dtype` field to both `Exp` and `Var` classes to track the type of every expression and variable reference. This field holds a reference to a declaration node, which contains the actual type information.

For expressions, I implemented type checking in the visitor methods. Integer literals get type int, and boolean literals get type bool. Variable references look up the variable in the symbol table and inherit its type. For array indexing, the result type is the element type of the array, not the array type itself. I implemented type rules for all operators: arithmetic requires integers, relational produces booleans, and equality checks matching types. Boolean operators (&&, ||, !) are more flexible. The C- specification allows both int and bool operands for these operators, treating any non-zero integer as true. The result is always a bool. Unary minus requires an integer operand and produces an integer result. Assignment type checking compares the

left-hand side and right-hand side types. They must match, with one special case: array parameters. In C-, when you declare a function parameter as `int a[]`, it can accept an argument of `type int[10]`. This is structural equivalence for arrays. I implemented a `typesMatch` helper function that handles this case by checking if both types are arrays with the same element type, regardless of size.

Function calls required the most complex type checking. I had to verify three things: the function exists, the argument count matches the parameter count, and each argument type matches the corresponding parameter type. For the count check, I handle the special case of void parameters (represented as a single `SimpleDec` with an empty name and void type), which means zero arguments. For argument type checking, I walk through the parameter list and argument list in parallel, comparing types. Array parameters again use structural equivalence. If an argument type doesn't match its parameter type, I report which argument number is wrong and what function it's for. Return statements check that the returned expression type matches the function's return type. Void functions can't return a value, and non-void functions must return a value of the correct type. I track the current function being analyzed with a currentFunction field, so return statements know what type is expected. Control flow statements (if and while) have a special rule in C-. The test condition can be either int or bool, but not void. This is more permissive than some languages but matches C-'s semantics. I check that the test expression isn't void and accept anything else.

Error Recovery and Type Inference

When semantic errors occur, I don't want the compiler to crash or stop analyzing. Error recovery lets me continue checking and find multiple errors in one run. My approach is to report the error with a descriptive message, then infer a sensible type so analysis can continue.

For undefined variables, I report the error and assign a dummy int type. This prevents cascading errors where every use of that variable causes another error. For type mismatches in operations, I report the specific mismatch (which operand, which operator) and infer the result type based on what the operator should produce. For example, if someone writes `bool + int`, I report the error but still give the expression type int, because that's what addition produces. I created two dummy declarations (`dummyInt` and `dummyBool`) that get reused throughout type checking. These are `SimpleDec` nodes with no name and the appropriate type. Using shared dummy declarations is more efficient than creating new ones for every error.

One challenge was preventing semantic analysis when there are syntax errors. If the parser encountered errors, the AST is incomplete or malformed, and semantic analysis would produce meaningless results or crash. I added a static `hasParseErrors` flag to the parser that gets set whenever an error occurs. The main driver checks this flag and skips semantic analysis if it's true.

Testing

I created five test programs to cover different aspects of semantic analysis. **`test/1.cm`** validates clean compilation with all C- features. **`test/2-4.cm`** each contains exactly three errors, focusing on symbol table issues, type checking, and function calls, respectively. **`test/5.cm`** is a comprehensive stress test with 18 errors of various types, testing error recovery. **All testing was done on both my machine (macOS) and the school's Linux server with identical results**.

Lessons Learned

The visitor pattern proved to be the right choice for semantic analysis. It kept the symbol table and type-checking logic cleanly separated from the AST node definitions. Adding new checks or

modifying existing ones was straightforward because all the logic is in one class. Post-order traversal was essential. I needed to check the children before the parents so that the expression types are known bottom-up. For example, to type-check x + y, I first need to visit x and y to determine their types, then check if those types are compatible with addition.

Error recovery is harder than I expected. It's not enough to just report an error. You have to keep the compiler in a consistent state so it can continue analyzing. Type inference helped a lot here. By assigning sensible default types when errors occur, I prevented cascading errors that would confuse users. Scope tracking with level numbers was simpler than I thought it would be. I initially considered more complex data structures, but just incrementing an integer and comparing levels worked perfectly. The key insight was that you only need to track the current level and compare it to stored levels, not maintain a full scope stack.

Testing incrementally saved me a lot of debugging time. I got basic symbol table insertion and lookup working first, tested it thoroughly, then added type checking for simple expressions, tested that, then added function checking, and so on. Each step was built on verified functionality.

Limitations and Future Work

My type checker accepts all valid C- programs and rejects invalid ones as required by the specification. Error messages are clear and functional, though they could be enhanced with more specific type information. The next step for checkpoint three is code generation, extending the visitor to generate TM assembly code.

Conclusion

I now have a complete front-end for C- that performs full semantic analysis. The symbol table correctly tracks declarations across nested scopes, handles function parameters, and provides the "most closely nested" lookup semantics. The type checker verifies operations, assignments, function calls, and control flow statements according to C-'s rules.

The compiler has been thoroughly tested with both hand-written test cases and the provided sample programs. It works correctly on both macOS and the Linux server with identical results on both platforms. The build/make has a few deprecation warnings in the CUP-generated code when using Java 17+, but this does not affect any functionality and was all expected.